

Deep Learning for Language: Recurrent Neural Networks

Robin Jia
USC CSCI 467, Fall 2026
October 7, 2026

Peculiarities of language data

- Peculiarity #1: Text is not a numerical format
 - Feature vector = list of numbers
 - Image = $3 \times W \times H$ grid of pixel brightness values
 - Text = sequence of words, not numbers
- Peculiarity #2: Text is variable sized
 - Feature vectors are always the same size for different examples
 - Images can be cropped/rescaled to be the same size for all examples
 - Text: Different examples have different # of words

Feeding Words to a Neural Network

- Peculiarity #1: Words are not numerical
- Solution: Learn word vectors, feed word vector of each word to model!
- Original input: T words
- Vector input: T vectors, each of size d

Text input: *A* *zoo* *elephant*

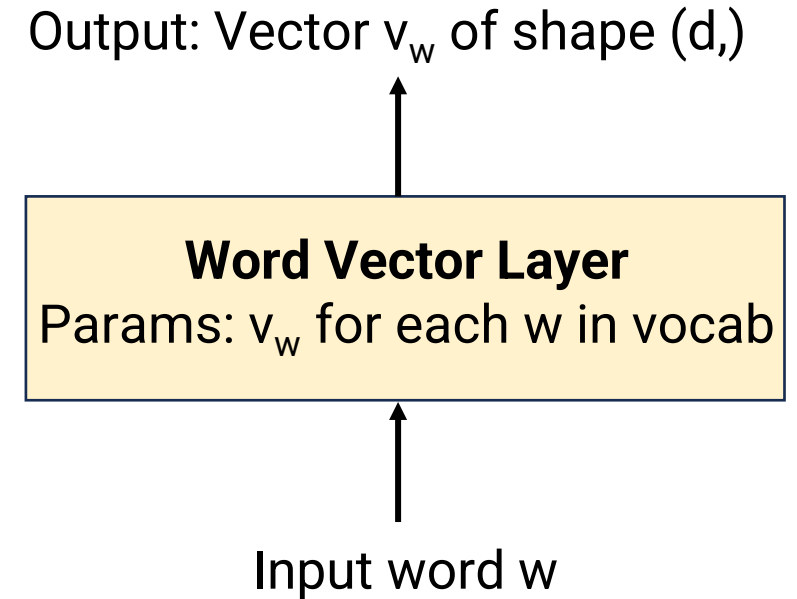
Vector input:	-0.4	2.1	2.1
	1.4	-1.4	-1.3
	-1.2	3.2	0.3

Word w	Vector v_w
A	[-0.4, 1.4, -1.2]
Aardvark	[2.2, -1.8, 0.6]
Airport	[0.7, 0.3, 3.1]
...	
Elephant	[2.1, -1.3, 0.3]
...	
Zoo	[2.1, -1.4, 3.2]

RNN “Building Blocks”

(6) Word Vector Layer

- Input w : A word (from our vocabulary)
 - Can also input list of words
- Output: A vector of length d
 - If input is many words, output is list of vectors for each word
- Formula: Return `word_vecs[w]`
- Parameters:
 - For each word w in vocabulary, there is a word vector parameter v_w of shape d
 - Think of this as a dictionary called `word_vecs`, where the keys are words & values are learned parameter vectors
 - Can initialize using `word2vec`, or randomly
 - Train them further with gradient descent to help final task
- In pytorch: `nn.Embedding()`

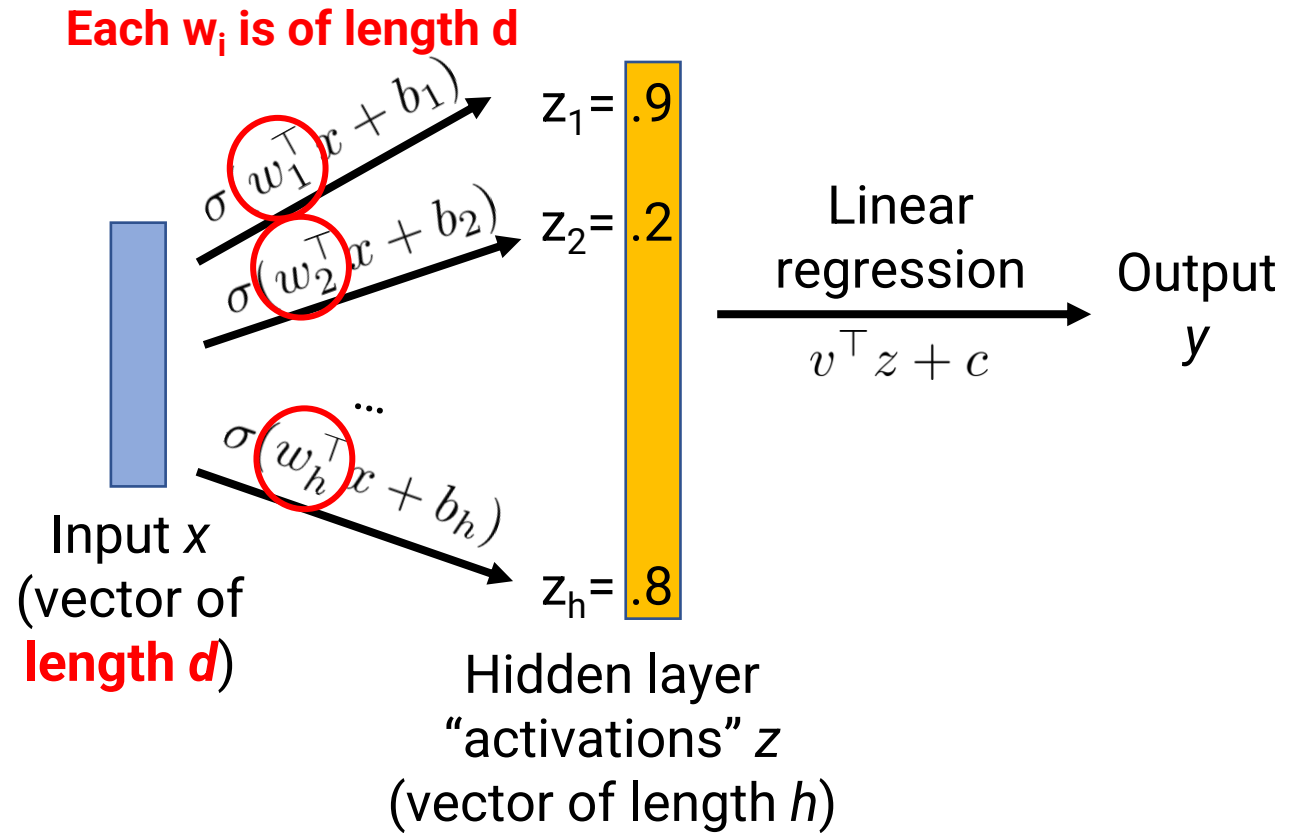


Outline

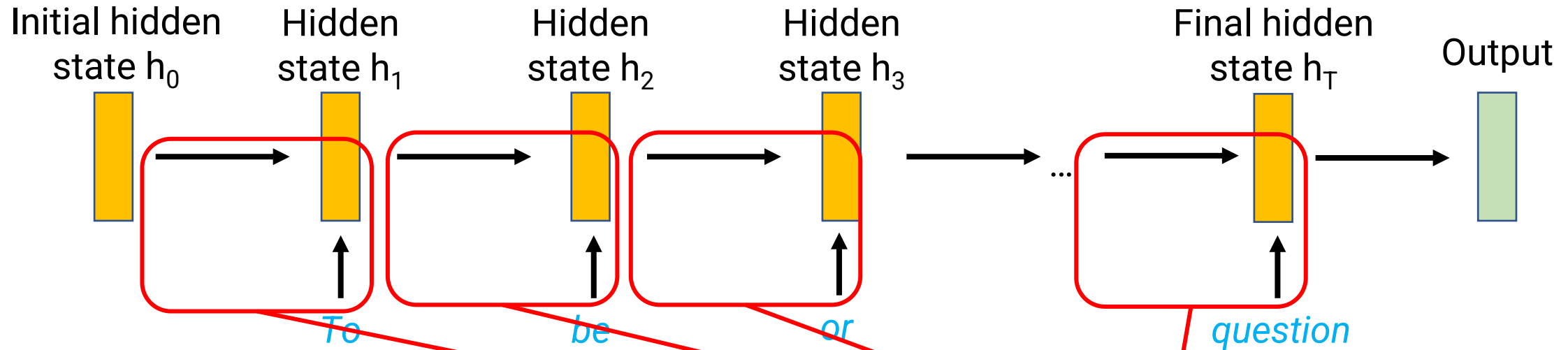
- Recurrent Neural Networks (RNNs) for sequential data
- Language modeling and Long-range dependencies
- Vanishing gradients and Gated RNNs

Handling variable length

- Peculiarity #2: Documents have different numbers of words
 - Example 1: *Amazing!*
 - Example 2: *There are many issues with this movie, such as...*
- Problem: In previous models, number of parameters depends on size of inputs
- Challenge: How can we use the **same** set of model parameters to handle inputs of any size?

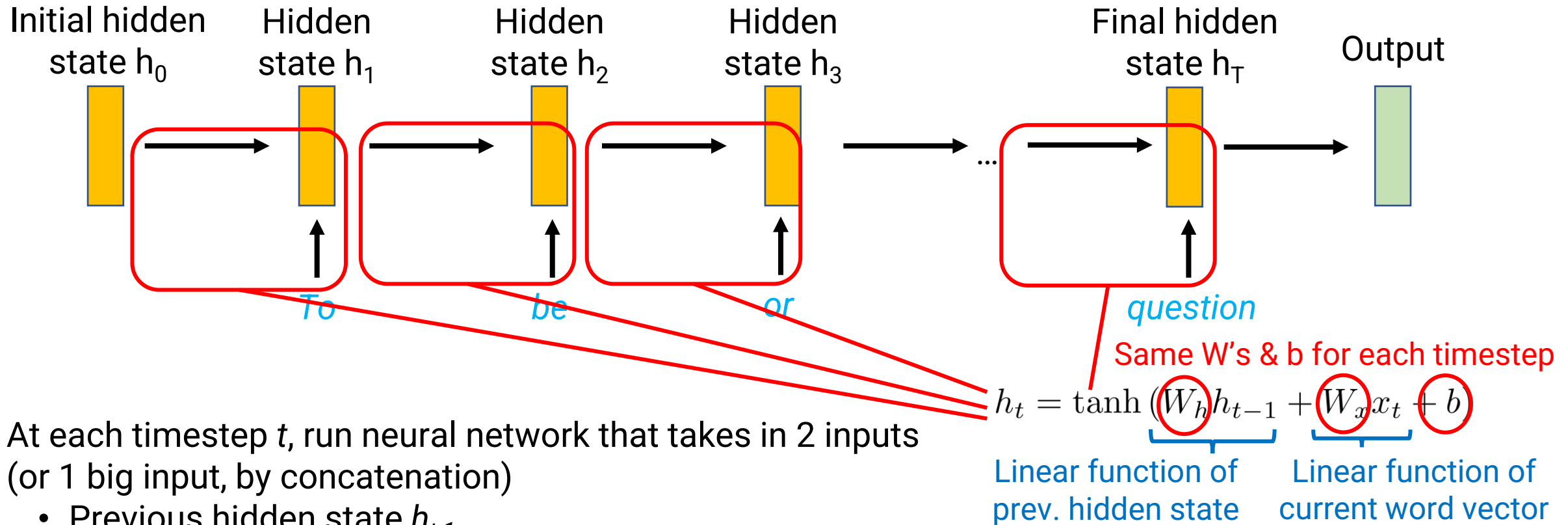


Recurrent Neural Networks (RNNs)



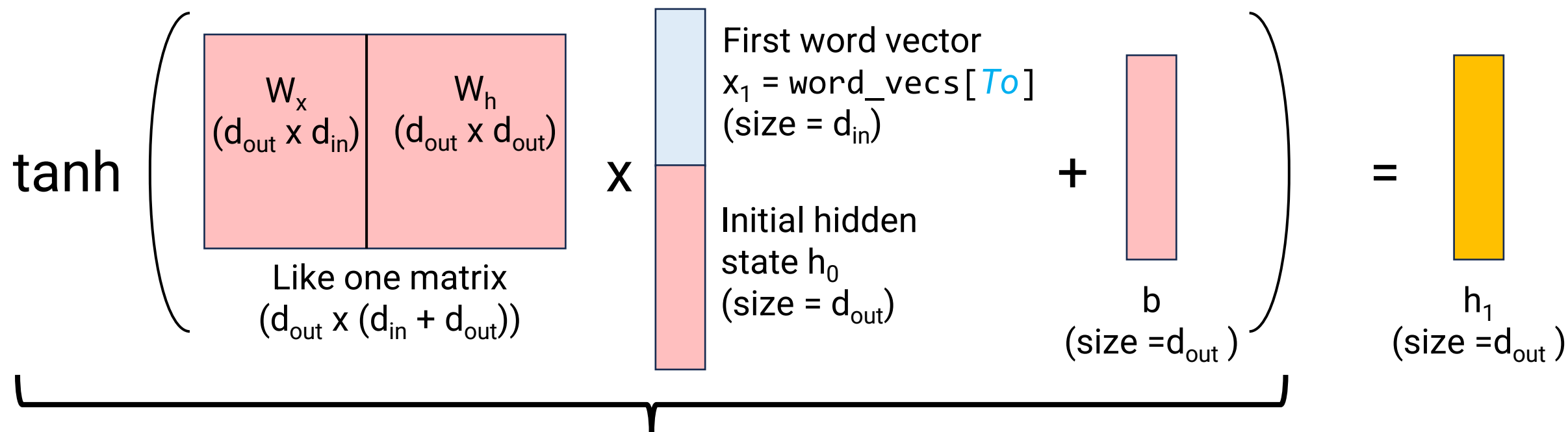
- Idea: Recurrence!
 - “Read” the input one word at a time
 - At each step, update the hidden state of the network
 - **Model parameters to do this update are same for each step**
- Each step is an application of the **same** neural network

A “Vanilla”/“Elman” RNN



- At each timestep t , run neural network that takes in 2 inputs (or 1 big input, by concatenation)
 - Previous hidden state h_{t-1}
 - Vector for current word x_t
- Learn linear function of both inputs, add bias, apply non-linearity
- Parameters: Recurrence params (W_h , W_x , b), initial hidden state h_0 , word vectors

RNN as Repeated Application of 1 MLP layer



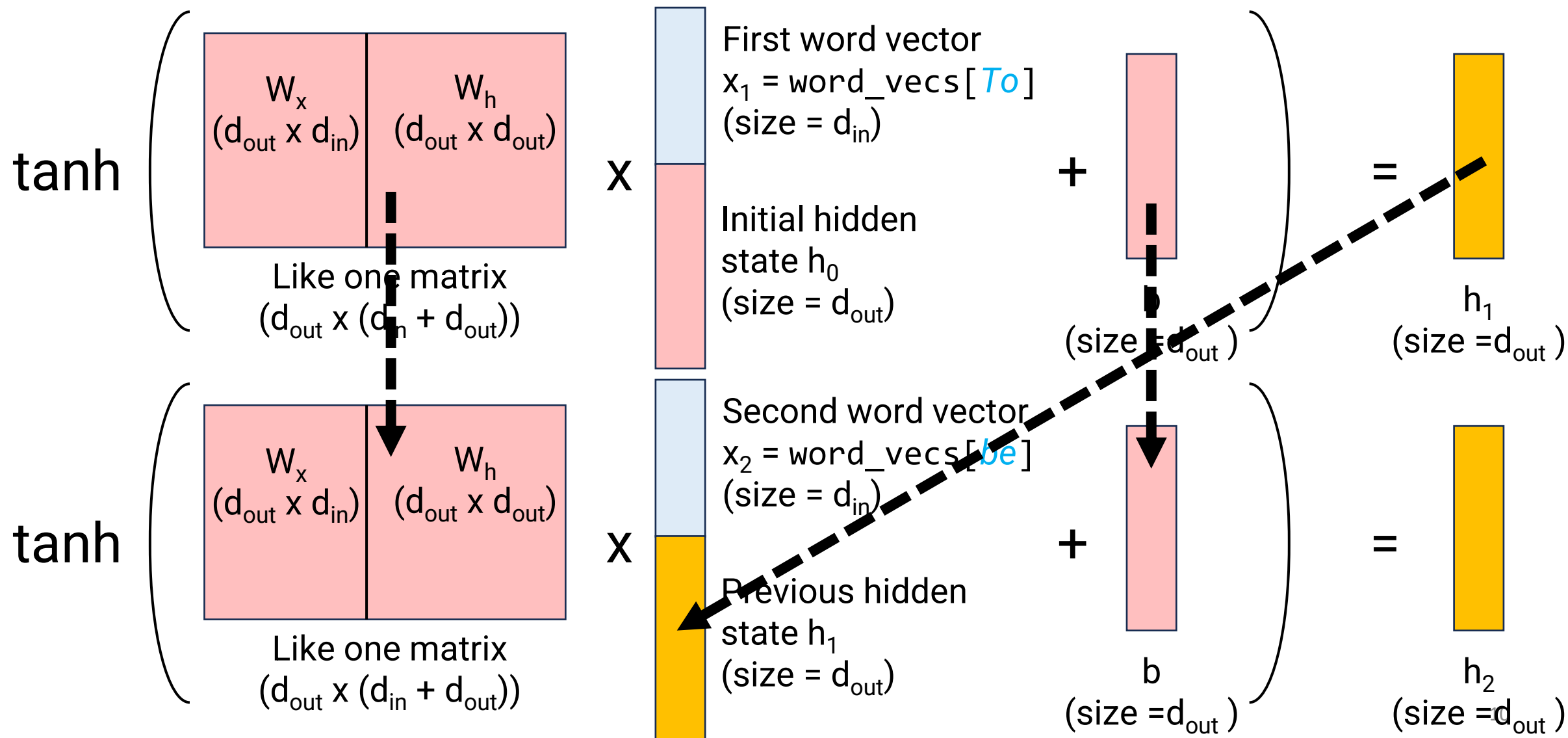
Legend:

Word vectors

Other learned parameters

Hidden states

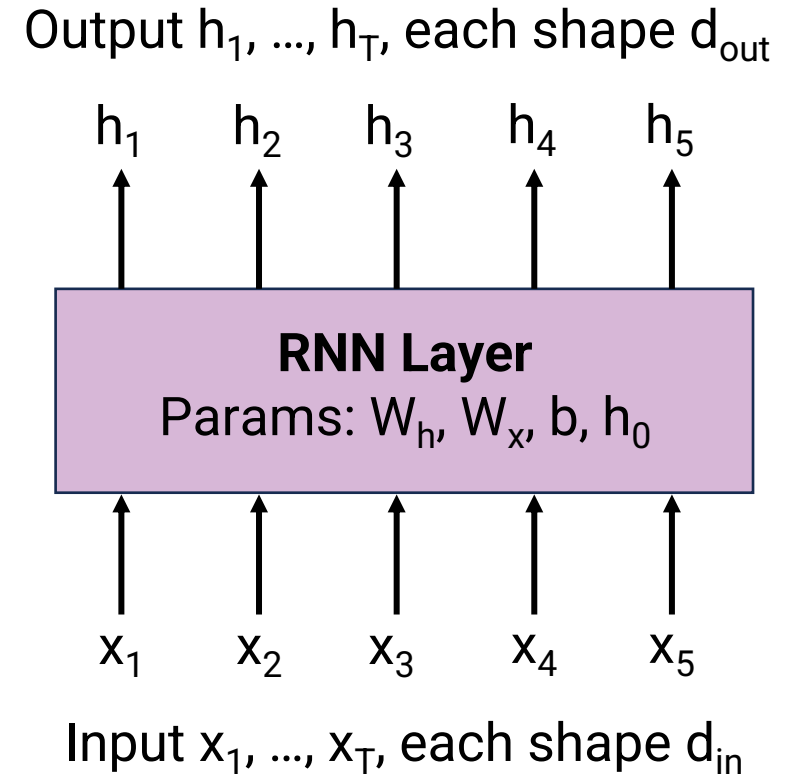
RNN as Repeated Application of 1 MLP layer



RNN “Building Blocks”

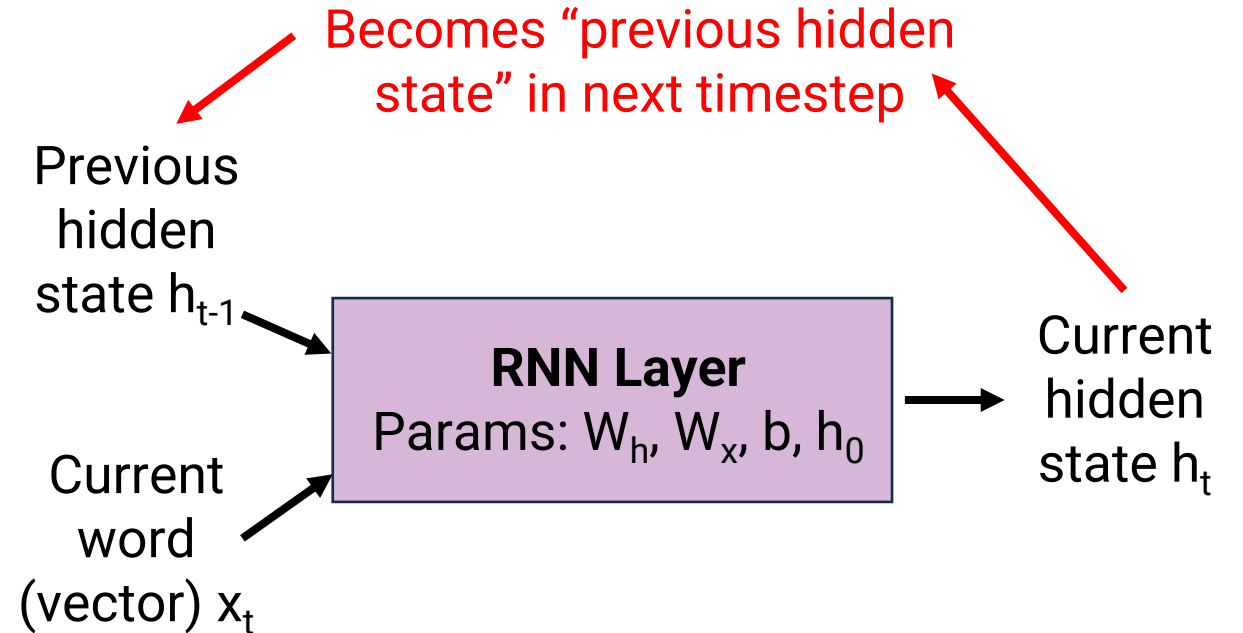
(7) RNN Layer

- Input: List of vectors $x_1, \dots, x_T$, each of size d_{in}
 - E.g., x_t is word vector for t -th word in sentence
 - Equivalent to a $T \times d_{in}$ matrix
- Output: List of vectors $h_1, \dots, h_T$, each of size d_{out}
 - d_{out} : Dimension of hidden state
 - Equivalent to a $T \times d_{out}$ matrix
- Formula (Elman RNN): $h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$
- Parameters:
 - W_h : Matrix of shape (d_{out}, d_{out})
 - W_x : Matrix of shape (d_{out}, d_{in})
 - b : Vector of shape (d_{out})
 - h_0 : Vector of shape (d_{out})
- In pytorch: `nn.RNN()`, etc.

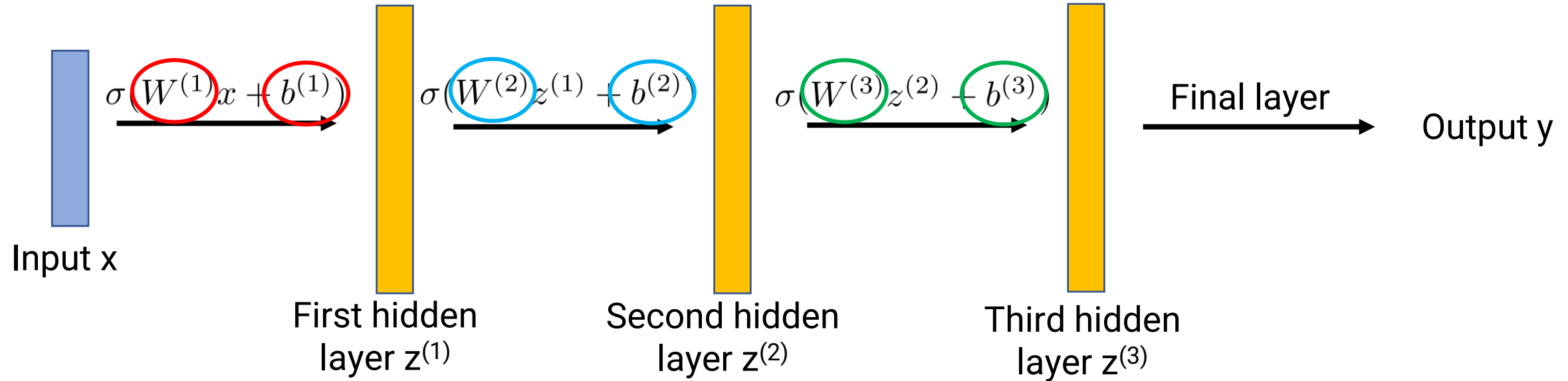


Recurrent Neural Network Diagrams

- Can also visualize RNN's with a diagram that has a cycle ("recurrence")
 - RNN layer is just a neural network that takes in two vectors and produces a third vector
 - New vector gets fed back in next timestep
 - Previous slide is the "unrolled" diagram

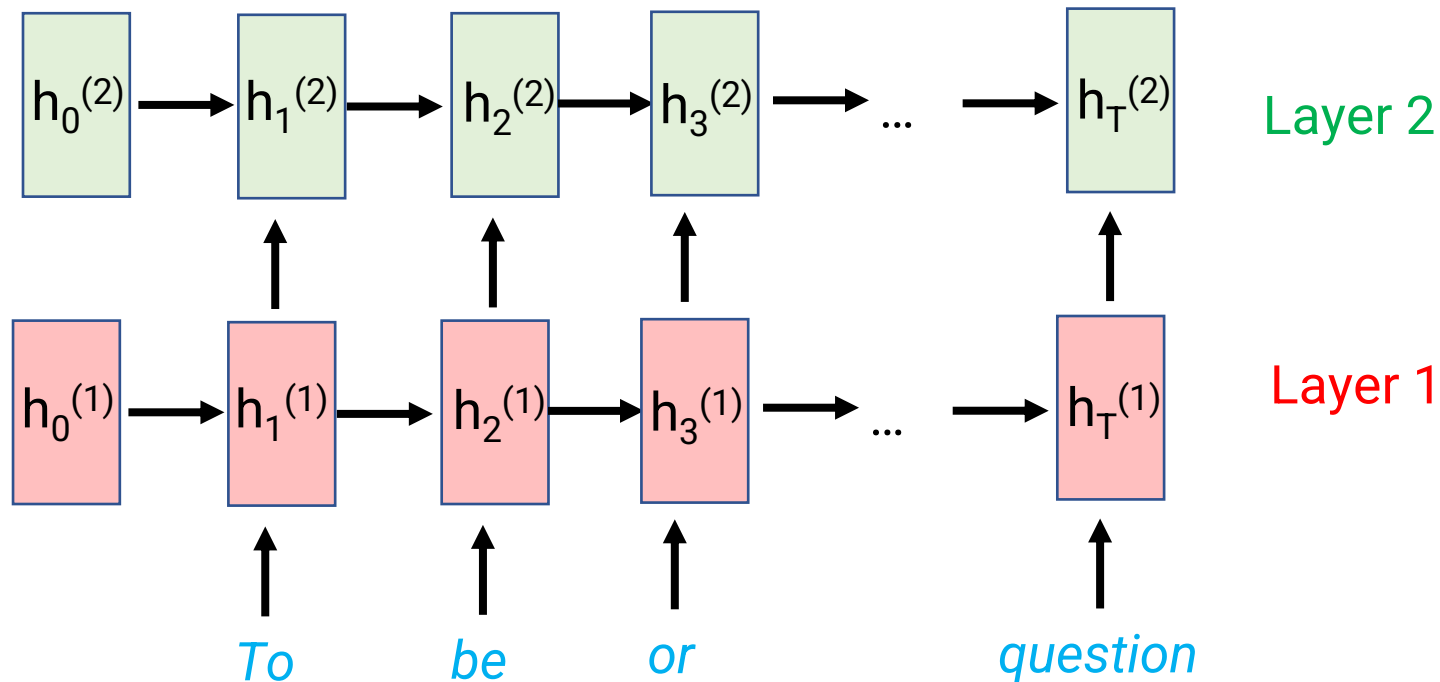


Recurrence vs. Depth



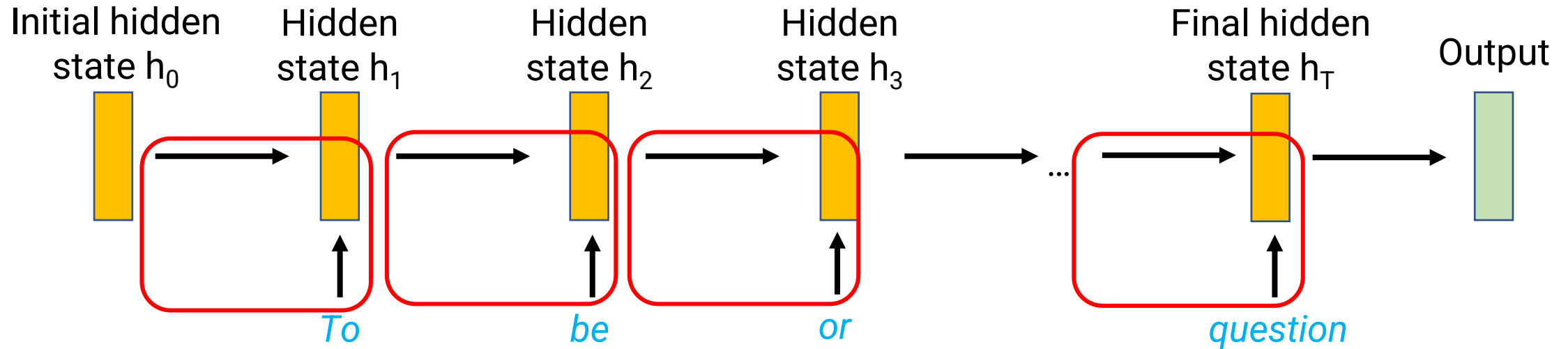
- Deep networks (i.e., adding more layers)
 - Computation graph becomes longer
 - Number of parameters also grows; each step uses new parameters
- Recurrent neural networks
 - Computation graph becomes longer
 - Number of parameters **fixed**; each step uses **same parameters**

Recurrence and Depth



- You can have multiple layers of recurrence too!
 - Left-to-right axis (“time dimension”): Length is size of input, same parameters in each step
 - Top-to-bottom axis (“depth dimension”): Length is depth of network, different parameters in each row

Training an RNN

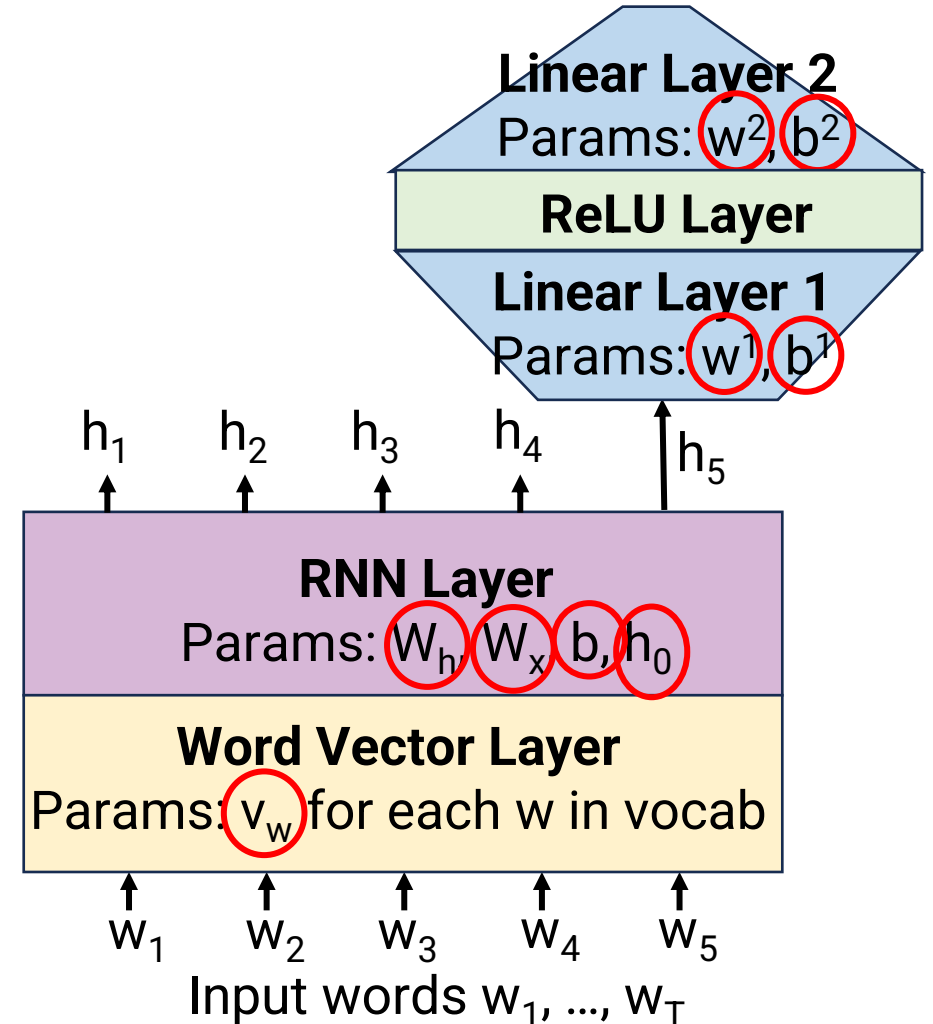


- Basic usage of RNN: Make prediction based on final hidden state
- Same recipe: Backpropagation to compute gradients + gradient descent
- Must backpropagate through whole computation graph
 - “Backpropagation through time”

Building an RNN encoder model

- A generic RNN architecture
 - Map each word to a vector
 - Feed word vectors to RNN to generate list of hidden states
 - Feed final hidden state to MLP to make final prediction (e.g., document classification)
- Basic steps are still all the same
 - Backpropagation still works
- Gradient descent needed to update all parameters

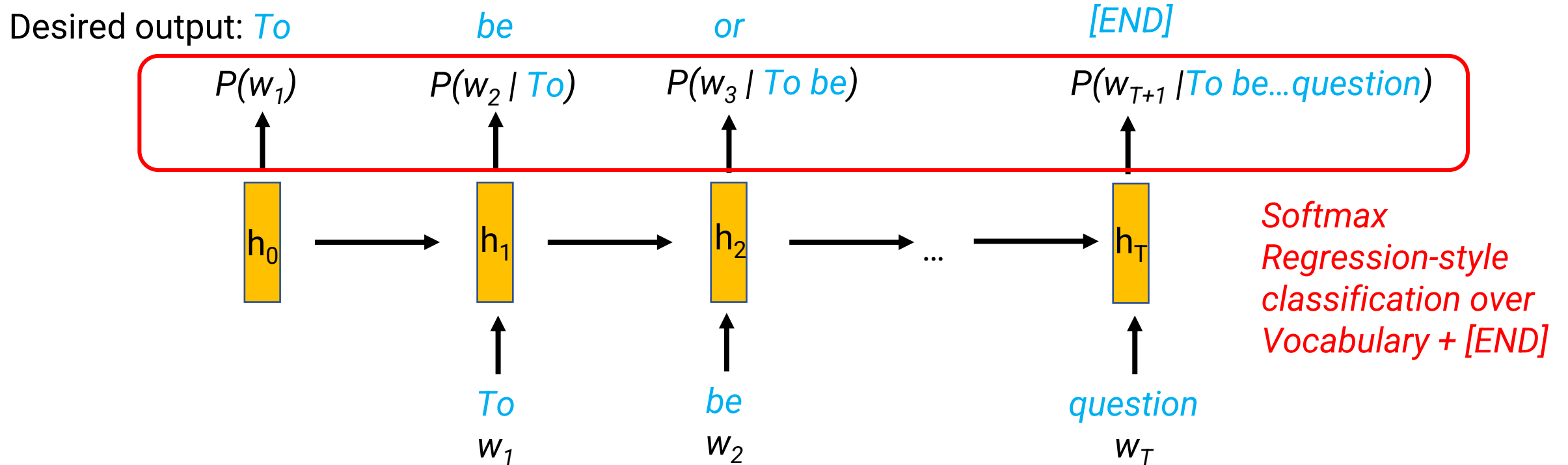
Neural Network Model



Outline

- Recurrent Neural Networks (RNNs) for sequential data
- Language modeling and Long-range dependencies
- Vanishing gradients and Gated RNNs

Autoregressive Language Modeling

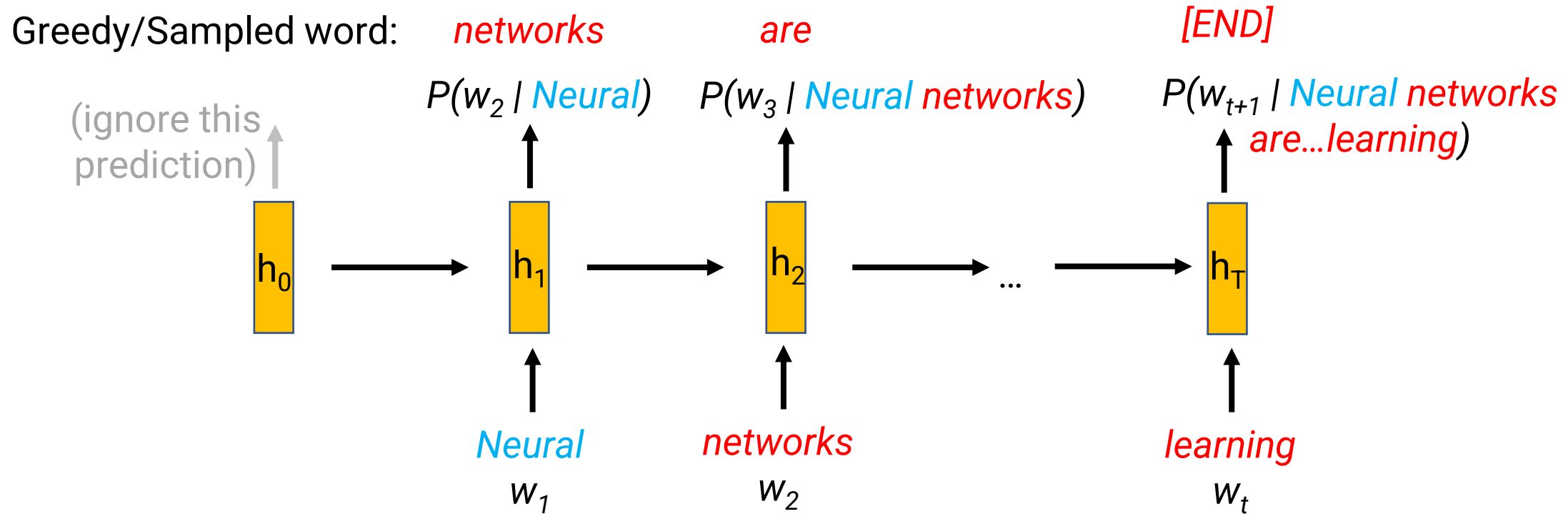


- At each step, probabilistically predict the next word given current hidden state
- One step's desired output is the next step's input ("autoregressive")
- To mark end of sequence, model should predict the *[END]* token
- Called a "Decoder": Looks at the hidden state and "decodes" next word

Autoregressive Language Model Training

- Training example: “Convolutional neural networks are good for image classification”
- Want to maximize $P(\text{“Convolutional neural networks are good for image classification”})$
- MLE: Take log and decompose by chain rule:
$$\begin{aligned} &\log P(\text{“Convolutional”}) \\ &+ \log P(\text{“neural”} \mid \text{“Convolutional”}) \\ &+ \log P(\text{“networks”} \mid \text{“Convolutional neural”}) \\ &+ \log P(\text{“are”} \mid \text{“Convolutional neural networks”}) + \dots \end{aligned}$$
- Decomposes into a bunch of **next-word-classification** problems
- Backpropagation + gradient descent to minimize loss
 - Update RNN parameters
 - Update word vectors
 - Update final layer classifier over vocabulary

Generating text with LM's



- Test time: Given some *prefix*, “*autocomplete*” the rest of the sentence
- First, feed prefix as input to model
- At each timestep, choose next word based on model's predictions
 - Greedy: Choose the most likely word
 - Sampling: Sample from the model's probability distribution over words
- Feed the model's generated word back as the next word, stop if *[END]*

Long-Range Dependencies

- Every step, you update the hidden state with the current word
- Over time, information from many words ago can easily get lost!
- This means RNNs can struggle to model **long-range dependencies**

The keys to the cabinet ____ (on the table)
plural singular

Long-Range Dependencies

- Every step, you update the hidden state with the current word
- Over time, information from many words ago can easily get lost!
- This means RNNs can struggle to model **long-range dependencies**

*The **keys** to the cabinet **are** (on the table)*
plural singular

Long-Range Dependencies

- Every step, you update the hidden state with the current word
- Over time, information from many words ago can easily get lost!
- This means RNNs can struggle to model **long-range dependencies**

*The **keys** to the cabinet by the door **are** (on the table)*

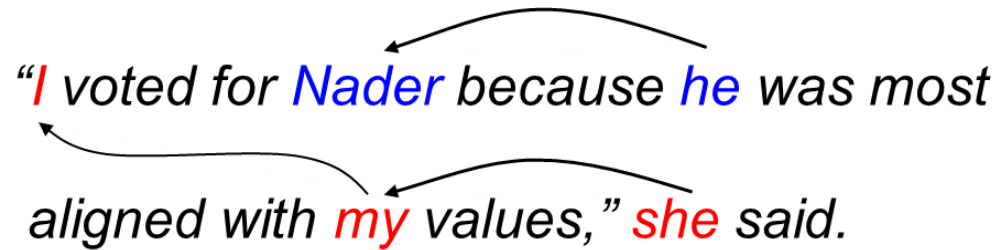
Long-Range Dependencies

- Every step, you update the hidden state with the current word
- Over time, information from many words ago can easily get lost!
- This means RNNs can struggle to model **long-range dependencies**

*The **keys** to the cabinet by the door on the left **are** (on the table)*

Long-Range Dependencies

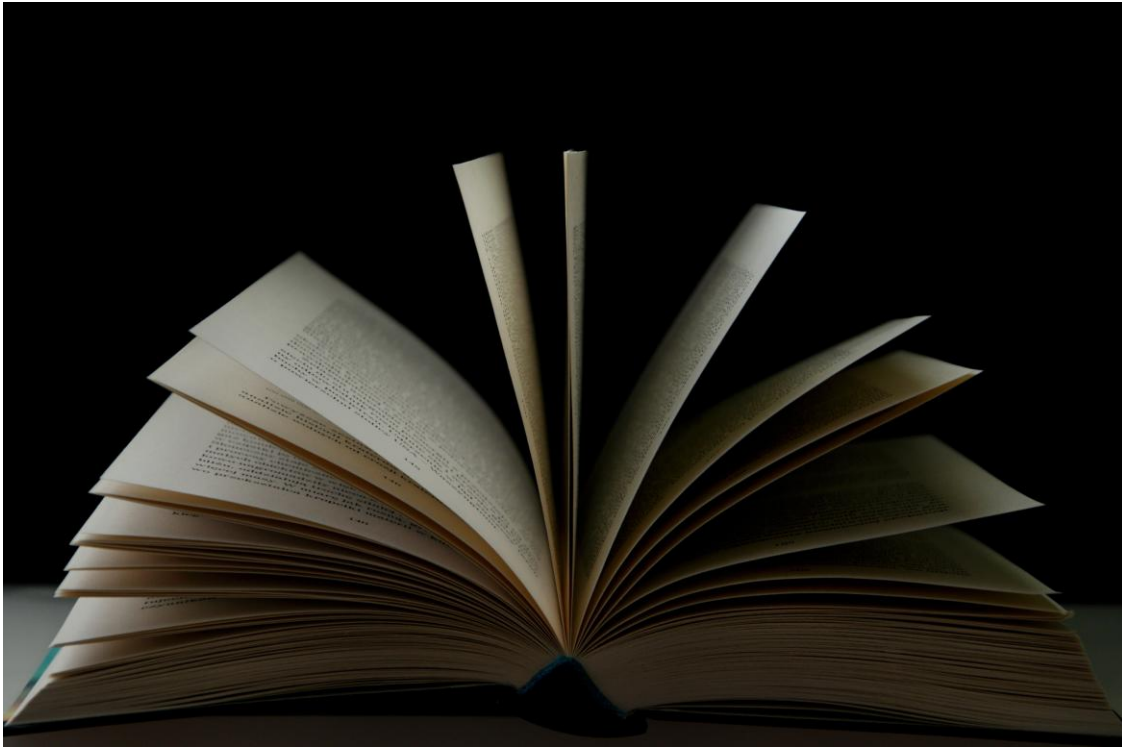
"I voted for Nader because he was most aligned with my values," she said.



The diagram illustrates coreference relationships in the sentence "I voted for Nader because he was most aligned with my values," she said. Three curved arrows indicate these relationships: one from "I" to "she", one from "Nader" to "he", and one from "my" to "values".

- “Coreference”: When two words refer to the same underlying person/place/thing
 - Pronouns typically **corefer** to an **antecedent** (something mentioned earlier in the text)
- Coreference relationships can even span multiple sentences

Even longer-range dependencies



- Imagine trying to generate a novel...
 - Same set of characters
 - Characters have to behave in consistent ways
 - Sensible ordering of events

Announcements

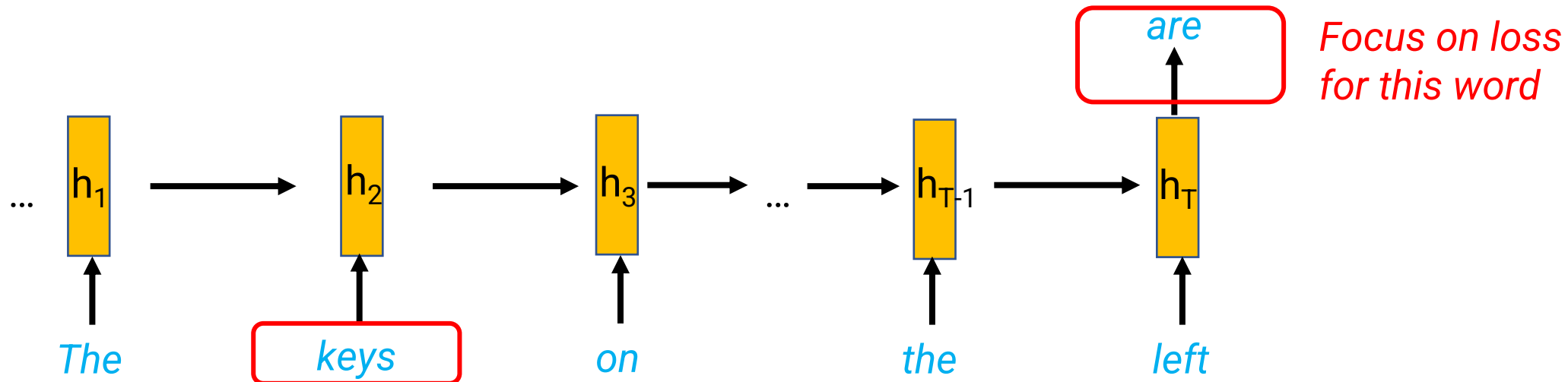
- HW0 grades out
 - Regrade requests open through next Monday, October 12
 - Solutions posted on Brightspace
- HW2 due next Wednesday, October 14
- Midterm exam Monday, October 19
 - Practice midterms posted online
 - Single sheet of 8.5"x11" paper of notes is allowed
 - No other aids are allowed

Outline

- Recurrent Neural Networks (RNNs) for sequential data
- Language modeling and Long-range dependencies
- **Vanishing gradients and Gated RNNs**

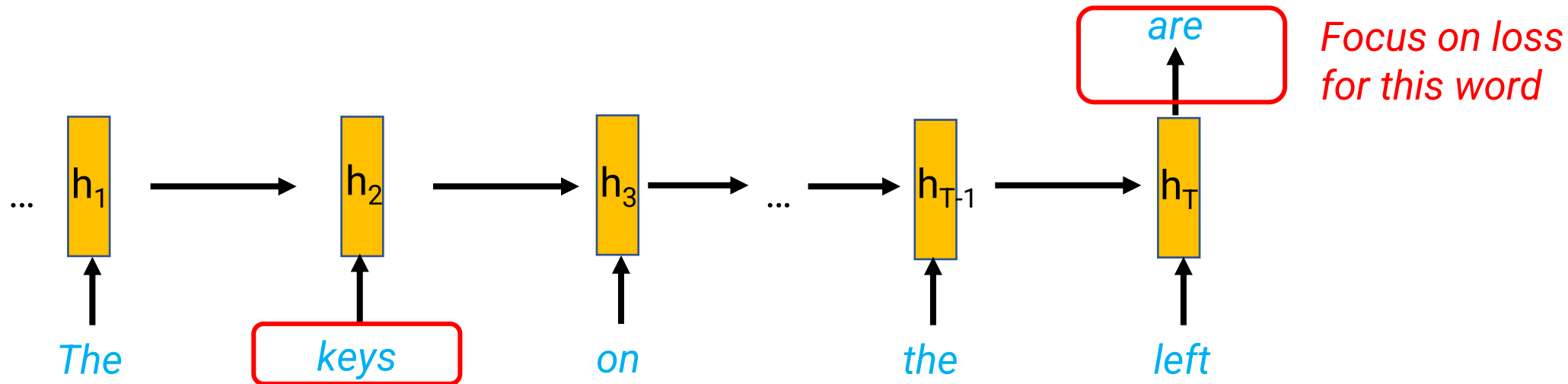
Backpropagation through time, revisited

- Model needs to know that the correct word is *are* because of the word *keys*!
- Let's backpropagate the loss on generating *are* to the word vector parameters for *keys*
 - For simplicity, let's assume all the hidden states are just 1-dimensional
 - Step 1: Compute $\delta\text{Loss}/\delta(h_T)$
 - Step 2: Compute $\delta\text{Loss}/\delta(h_{T-1}) = \delta\text{Loss}/\delta(h_T) * \delta(h_T)/\delta(h_{T-1})$
 - Step 3: Compute $\delta\text{Loss}/\delta(h_{T-2}) = \delta\text{Loss}/\delta(h_T) * \delta(h_T)/\delta(h_{T-1}) * \delta(h_{T-1})/\delta(h_{T-2})$
 - ...
 - Gradient through "*keys*" hidden state: $\delta\text{Loss}/\delta(h_T) * \delta(h_T)/\delta(h_{T-1}) * \delta(h_{T-1})/\delta(h_{T-2}) * \dots * \delta(h_3)/\delta(h_2)$
 - Gradient through "*keys*" word vector: $\delta\text{Loss}/\delta(h_T) * \delta(h_T)/\delta(h_{T-1}) * \delta(h_{T-1})/\delta(h_{T-2}) * \dots * \delta(h_3)/\delta(h_2) * \delta(h_2)/\delta(x_2)$



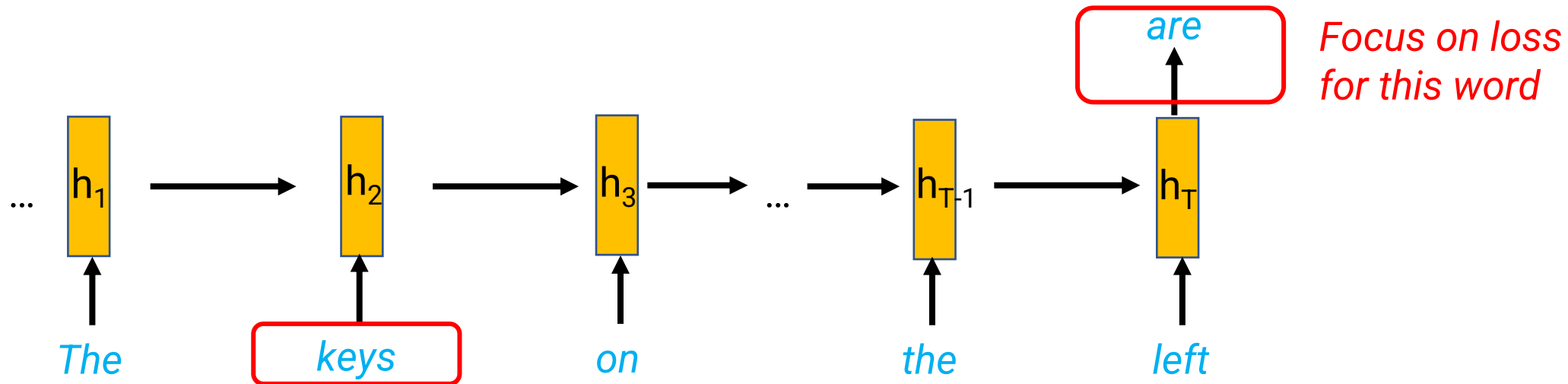
The Vanishing Gradient Problem

- Gradient through “*keys*” word vector: $\delta \text{Loss} / \delta(h_T) * \delta(h_T) / \delta(h_{T-1}) * \delta(h_{T-1}) / \delta(h_{T-2}) * \dots * \delta(h_3) / \delta(h_2) * \delta(h_2) / \delta(x_2)$
 - What is each individual $\delta(h_t) / \delta(h_{t-1})$ term?
 - Elman network: $h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$, $\frac{\delta h_t}{\delta h_{t-1}} = \underbrace{\tanh'(W_h h_{t-1} + W_x x_t + b)}_{\text{Ignore for now}} \cdot \underbrace{W_h}_{\text{The same parameter over and over!}}$
 - After t timesteps, have a factor of $(W_h)^t$ (to the t -th power)!
 - If $W_h \ll 1$, this quickly becomes 0 (“vanishes”)



The Vanishing Gradient Problem

- Vanishing Gradients: Updates to one word/hidden state not influenced by loss on words many steps in the future
 - Illustrated only for 1-dimensional hidden states, but same thing happens when states are vectors/parameters are matrices
- Result: Hard for model to learn long-range dependencies!



Vanishing and Exploding

- Vanishing gradient occurs because
 - Gradient w.r.t. words t steps in the past has $(W_h)^t$
 - And when $W_h \ll 1$ (e.g., at initialization time)
- What if $W_h > 1$?
 - Gradients get bigger as you go backwards in time: Exploding gradients!
 - Vanishing gradients more usual, but explosion can happen too
- Quick fix: Gradient Clipping
 - If gradient is super large, “clip” it to some maximum amount
 - Rescale the total vector to some maximum norm
 - Clip each entry to be within some minimum/maximum value
- Outside of RNNs, vanishing/exploding gradients can happen whenever you have long computation graphs with lots of multiplications

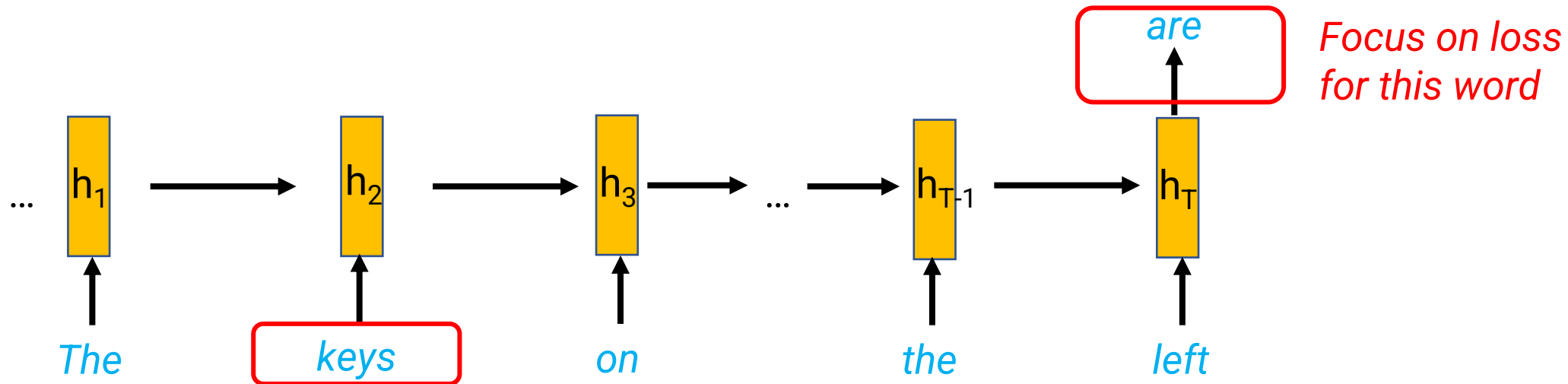
Avoiding Vanishing Gradients

- Where did we go wrong?

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b), \quad \frac{\delta h_t}{\delta h_{t-1}} = \tanh'(W_h h_{t-1} + W_x x_t + b) \cdot \underbrace{W_h}$$

Multiplicative
relationship between previous
state and next state

Leads to repeated
multiplication by W_h



Avoiding Vanishing Gradients

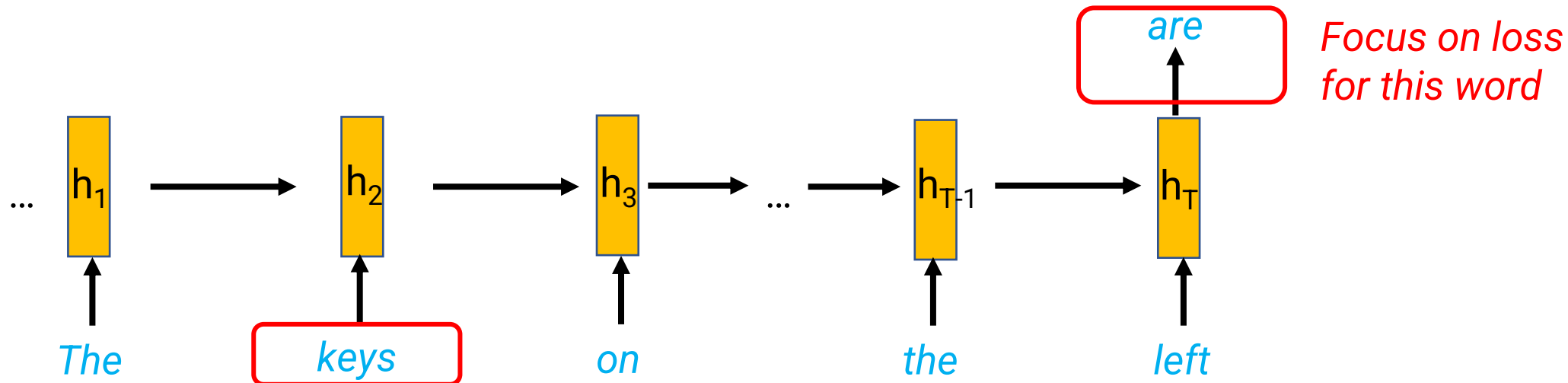
- Extreme idea: A purely additive relationship
 - Pro: No vanishing gradients
 - Pro: Old hidden state carried through to all future times
 - Con: May be good to “forget” irrelevant information about old states

$$h_t = h_{t-1} + \underbrace{g(h_{t-1}, x_t)}_{\text{Additive relationship}}$$

Additive relationship

$$\frac{\delta h_t}{\delta h_{t-1}} = 1 + \underbrace{\frac{\delta}{\delta h_{t-1}} g(h_{t-1}, x_t)}_{\text{Gradients also add, not multiply}}$$

Gradients also add, not multiply

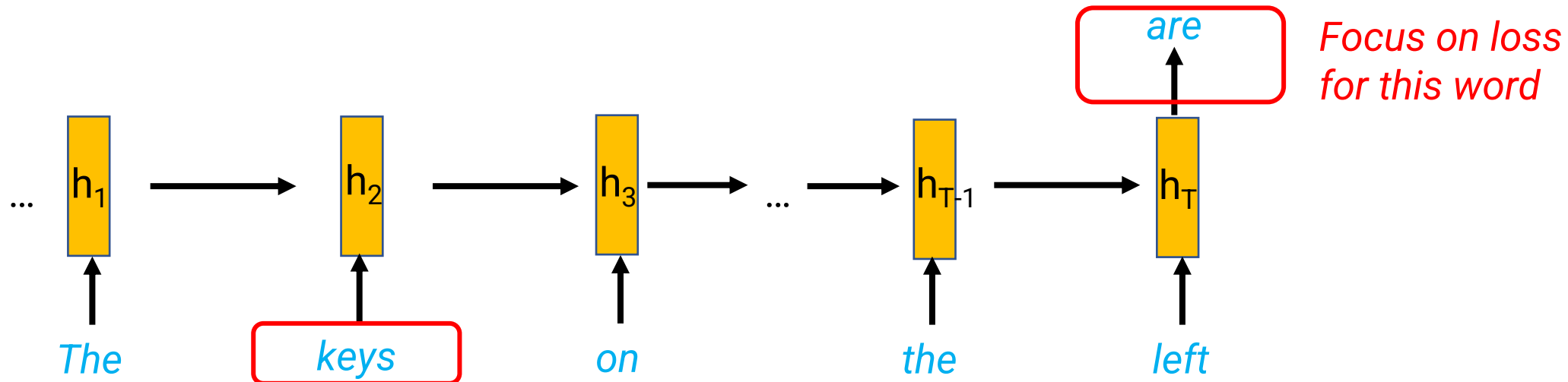


Avoiding Vanishing Gradients

- Middle-ground: **Gated** recurrence relationship
 - Additive component makes gradients add, not multiply = less vanishing gradients
 - Forget gate allows for selectively “forgetting” some neurons within hidden state
 - When forget gate is all 1’s, becomes the purely additive model (no vanishing)

$$h_t = h_{t-1} \odot \underbrace{f(h_{t-1}, x_t)}_{\substack{\text{“forget gate”} \\ \text{in } [0, 1]}} + \underbrace{g(h_{t-1}, x_t)}_{\substack{\text{Additive} \\ \text{relationship}}}$$

Elementwise multiplication

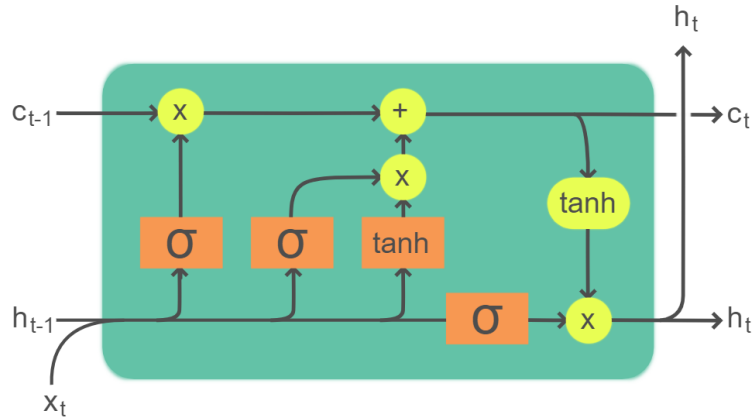


Gated Recurrent Units (GRUs)

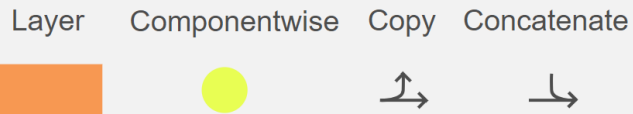
- One type of gated RNN
 - Here z is the “forget gate” vector
 - Where $z_i = 0$:
 - Forget this neuron
 - Allow updating its value
 - Where $z_i = 1$:
 - Don’t forget this neuron
 - Do not allow updating its value
- Parameters: W , U , plus parameters of g
 - (g has a slightly complicated form not shown, has its own parameters)

$$h_t = h_{t-1} \odot z + \overbrace{g(x_t, h_{t-1}) \odot (1 - z)}^{\text{Additive relationship}}$$
$$z = \underbrace{\sigma(Wx_t + Uh_{t-1})}_{\text{Sigmoid ensures gate is between 0 and 1}}$$

Long Short-Term Memory (LSTM)



Legend:



- Another, more complicated gated RNN
- Commonly used in practice
- What's the idea?
 - Split the hidden state into normal hidden state h_t and "cell" state c_t
 - Cell state uses gated recurrence
 - Hidden state is gated function of cell state

$$f_t = \sigma_g(W_f x_t + U_f h_{t-1} + b_f)$$

$$i_t = \sigma_g(W_i x_t + U_i h_{t-1} + b_i)$$

$$o_t = \sigma_g(W_o x_t + U_o h_{t-1} + b_o)$$

$$c_t = f_t \circ c_{t-1} + i_t \circ \sigma_c(W_c x_t + U_c h_{t-1} + b_c)$$

$$h_t = o_t \circ \sigma_h(c_t)$$

What do LSTMs learn?

- Here: a character-level LSTM (not word-level)
- Blue/Green: Low/high values of 1 neuron
- Below: Top-5 predictions for next character
- This neuron seems to detect whether we're inside a URL

t	t	p	:	/	/	w	w	w	.	y	n	e	t	n	e	w	s	.	c	o	m	/	]		E	n	g	l	i	s	h	-	l	a	n	g	u	a	g	e		w	e	b	s	i	t	e		o	f		I	s	r	a	e	l	'	s		I	a	r					
t	p	:	/	/	w	w	w	.	b	a	c	a	h	e	t	s	.	c	o	m	/			-	x	g	l	i	s	h		l	i	n	g	u	a	g	e	s		a	i	r	s	i	t	e		o	f		t	s		I	s	r	a	e	l	i	s		s	i	n	g	
	d	:	x	n	e	.	w	a	e	a	.	.	a	w	a	t	o	a	.	s		&	n	t	i	a	c	a	-	s	a	r	d	e	e	l	h		o	a	n		t	b	i	s	a	n	f	a	n	r	e	i	f		'		a	a	t	d							
m	w	-	2																				(	d	c	e	e	n		e	p	e	s	a	a	i	k	i		i	e	e	l	e	d	h	,	i	r	t	h	r	a	o	n	s	e		, c	o	s	e							
d	r	.	<	:	a	h	b	-	n	p	t	w	t	.	x	i		g	h	/	m	a	)	T	v	d	r	y	z	i		c	o	u	e	d	l	s	u	:	t	h	a	-	o	o		t	u	,	s	t	u	i	f		l	v	e	p	e	r	y						
s	t	p	,	t	c	o	a	2	d	r	u	l	w	o	c	l	e	n	s	r	]p.		l	i	v	a	o	d	,	,	e	y	t	c	-	n		d	m	-	o	i	b	u	v	s	]b	b		i	m	s	u	l	t	a		t	l	y	b	n							

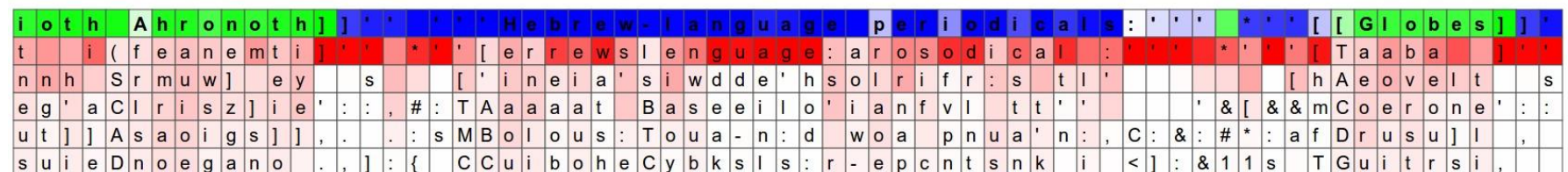
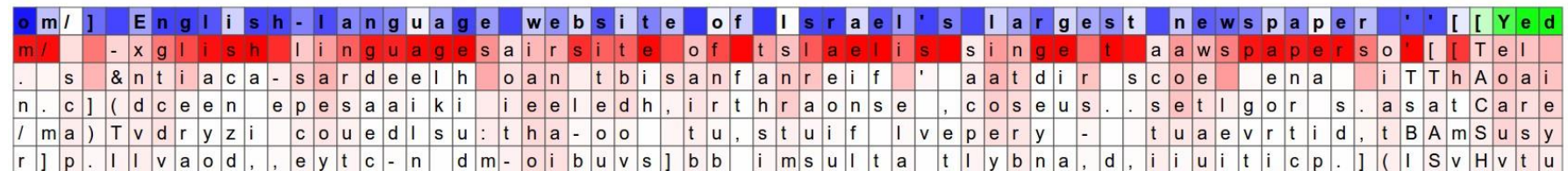
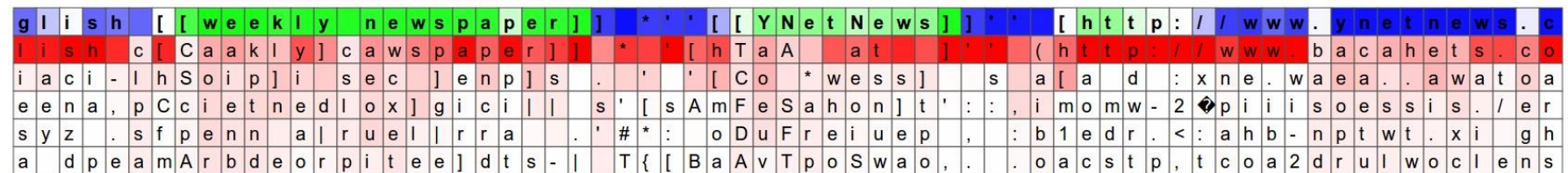
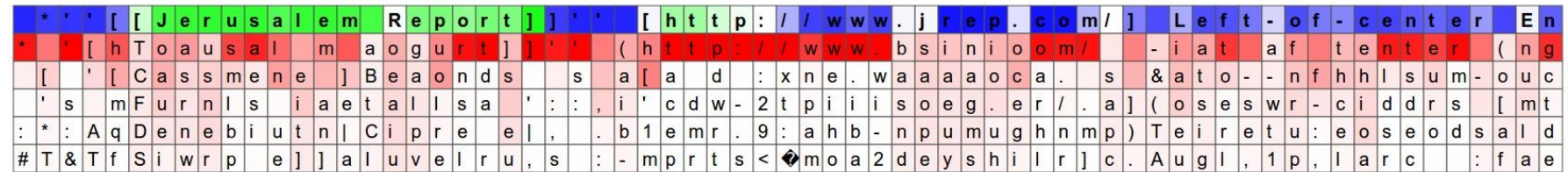
g	e	s	t		n	e	w	s	p	a	p	e	r		'	'	[	[	Y	e	d	i	o	t	h		A	h	r	o	n	o	t	h	]	]	'	'		'	'	H	e	b	r	e	w	-		l	a	n	g	u	a	n	g	e		p	e	r	i	o	d	
e	t		a	a	w	s	p	a	p	e	r	s	o	'	[	[	T	e	l	t		i	(	f	e	a	n	e	m	t	i	]	'	'	*	'	'	[	e	r	r	e	w	s	l	e	n	g	u	a	n	g	e	:	a	r	o	s	o	d	i					
i	r		s	c	o	e		e	n	a			i	T	T	h	A	o	a	i	n	n	h		S	r	m	u	w	]		e	y			s					[	'	i	n	e	i	a	'	s	i	w	d	d	e	'	h	s	o	l	r	i	f	r	:		
u	s	.	.	s	e	t	l	g	o	r		s	.	a	s	a	t	C	a	r	e	e	g	'	a	C	l	r	i	s	z	]	i	e	'	:	:	,	#	:	T	A	a	a	a	a	t		B	a	s	e	e	i	l	o	'	i	a	n	f	v	l			
		-			t	u	a	e	v	r	t	i	d	,	t	B	A	m	S	u	s	y	u	t	]	]	A	s	a	o	i	g	s	]	]	,		.	:	s	M	B	o	l	o	u	s	:	T	o	u	a	-	n	:	d		w	o	a		p	n	u		
a	,	d	,	i	i	u	i	t	i	c	p	.	]	(	I	S	v	H	v	t	u	s	u	i	e	D	n	o	e	g	a	n	o		.	,	]	:	{		C	C	u	i	b	o	h	e	C	y	b	k	s	l	s	:	r	-	e	p	c	n	t	s		

i	c	a	l	s	:	'	'	'	*	'	'	[	[	G	l	o	b	e	s	]	]	'	'	[	h	t	t	p	:	/	/	w	w	w	.	g	l	o	b	e	s	.	c	o	.	i	i	/	/	]	b	u	s	i	n	e	s	s	d	a								
c	a	l	:	:	'	'	'	*	'	'	'	[	T	a	a	b	a		]	'	'	'	(	[	t	t	p	:	/	/	w	w	w	.	b	u	o	b	a	l	.	c	o	m	u	n	/	s	A	-	y	t	i	n	e	s	s	a	e	t								
s	t	l	'									[	h	A	e	o	v	e	l	t		s		a	h	a	d	:	x	g	e	.	w	a	o	i	r	.	r	t	o	a	.	e	l	.	i	T		&	a	i	.	e	g			e	o	o	y							
t	t	'	'									'	&	[	&	&	m	C	o	e	r	o	n	e	'	:	:	,	i	'	o	d	w	.	,	:	n	i	i	s	a	a	u	e	.	e	n	i	/	o	m	l	c	C	.	(	e	f	t	g	i	r			i	i	u	
a	'	n	:	,	C	:	&	:	#	*	:	a	f	D	r	u	s	u	]	l	,		.	o	m	e	l		p	<	,	d	h	a	:	d	e	u	o	o	t	/	i	h	n	c	s	i	f	S	,		u	r	h	o	s	t	,	t	u	n						
n	k	i		<	]	:	&	1	1	s		T	G	u	i	t	r	s	i	,			:	b	a	c	m	r	-	x	t	p	o	b	-	g	r	e	s	i	s	l	e	r	l	n	a	f	a	D	]	l	o	s	p	t	a	d	,	i	f	r	m					

i	l	y	*	'	'	[	[	H	a	a	r	e	t	z		H	a	'	A	r	e	t	z	]	]	'	'		[	h	t	t	p	:	/	/	w	w	w	.	h	a	a	r	e	t	z	.	c	o	.	i	l	/	/		R	e	l	a	t	i	v	e
l	y	*	'	'	[	[	T	e	r	r	d	n		F	e	r	a	n	t	a	h	]	]	'	'		(	[	t	t	p	:	/	/	w	w	w	.	b	o	n	m	d	s	t	.	c	o	m	u	n	/	s	-	e	s	a	t	e	o	i			
r	e			'	'	h	A	i	l	n	n	t	t	e	H	a	l	s	r	c	n	o	l	'	'	s		a	h	a		d	:	x	n	e	.	w	a	a	m	r	t	d	h	e	o	h	.	o	l	.	c	&	o	p	i	n	i	v	e			
k	i	.	:	*	s	C	O	S	a	n	l	t		h	i	T	i	m	'	l	i	]e			:	,	i	m	c	d	w	-	2	◆	p	h	i	i	s	e	r	d	i	t	.	i	n	a	/	c	m	f	i	.	(	a	f	l	c	a	n	a		
d	s	-	!	[	t	B	T	C	o	m	m	g	d	]	]	W	o	n		a		a	e	,	:	.	b	a	e	r	r	.	<	t	a	i	b	-	d	u	l	c	n	n	c	/	a	r	n	e	s	i	]l	i	c	e	y	s	t	o				
n	d	s	#	&	:	G	I	D	u	v	c	c	s	a	o	S	u	c	i	t	e	l	]z	]	,	:	o	'	o	m	t	]	,	:	e	o	a	2	n	i	v	f	s	r	o	o	e	i	u	n	a	l	a	)		u	v	v	r	o				

What do LSTMs learn?

- Here: a character-level LSTM (not word-level)
- Blue/Green: Low/high values of 1 neuron
- Below: Top-5 predictions for next character
- This neuron fires only inside a Markdown `[[link]]` (so it knows when to close the square brackets?)



Conclusion

- Deep Learning for Language must deal with possibly long inputs
- RNNs handle arbitrarily long inputs with fixed number of parameters
- Need to handle long-range dependencies, but hard to learn due to vanishing gradients
- Gated RNNs (GRUs, LSTMs) can reduce vanishing gradient problems